

# DEVOPS :-

## Computer Networking :-

CN means connecting computers, servers and other devices together so they can communicate and share resources (like files, apps or databases etc).

## Why CN imp. for Devops :-

Devops is all about building, deploying & managing application in cloud / servers.

Networking in Devops is imp. because :-

### 1) Server need to communicate with each other :-

Your app needs to connect to a database server. Without networking, they can't communicate.

### 2) Cloud resources depends on networking :-

In AWS / Azure / GCP, we set up VPC, subnets, firewalls, security groups in these topics we use concepts of networking.

### 3) CI/CD pipelines use networking :-

Jenkins, Github Action or Github runners need internet / n/w access to fetch code, deploy to servers or push docker images.

### 4) Monitoring & Troubleshooting :-

Tools like prometheus / grafana need n/w access to collect metrics from servers.

As a devops engineer, you often debug why an app isn't reachable (port issue, firewall rules, DNS etc).

## How Internet works:-

Think internet as a huge postal system, but instead of letters, it delivers data packets (tiny chunks of info.).

### Suppose:-

- You type a website (www.google.com) in browser.
- Your browser needs the server's ip address. (google's server)
- It asks DNS (Domain Name System) to convert the domain names (www.google.com) into an IP addr. (3.4.5.6).

Your request travels through the n/w

- It goes from your device → wifi router → ISP (Internet Service Provider) → multiple intermediate routers → finally to google's server.

The server receives your request

- The server checks what you asked for (like you want google homepage)

The server sends back a response

- It sends HTML, CSS, Javascript files → broken into small data packets → travels back the same way to your computer.

Your browser rebuilds the packet into a webpage

- Browser takes those files and displays the webpage you see.



It is basically a client-server architecture (model)

Client (Browser)

Hey server, give me google homepage → this is a request (HTTP/HTTPS)

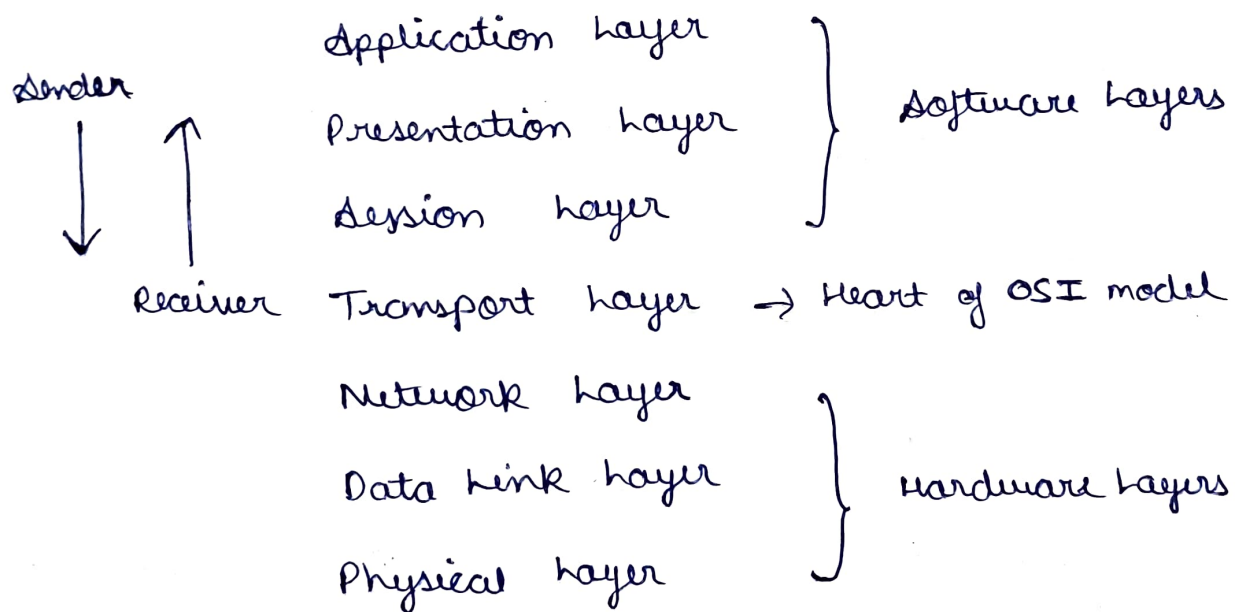
Server (Google's computer)

Here is the HTML page + images + scripts for your webpage → this is the response.

### OSI (Open System Interconnection) model :-

it is a conceptual framework that explains how data travels across a network - from one computer to another, by dividing the process into 7 layers each with specific functions.

#### 7 layers of OSI model:



#### 1) Physical layer:-

→ Deals with physical connections & transmits raw data as signals.

→ contains info. in form of bits.

Functions:- Bit Synchronization, Bit rate control, Topology, Transmission modes → Simplex, Half duplex, Full duplex.



## 2) Data link layer :-

- responsible for node to node delivery b/w two directly connected devices.
- uses MAC addr. (physical addr.) to identify devices.

It is divided into 2 layers:

- Logical link control → generally provides logic for data link.
- Media Access Control → MAC addr. (unique 48 bit)
  - \* MAC addr. is embedded into NIC during manufacturing.
  - \* MAC addr. uniquely identify n/w interfaces.

MAC addr. types  $\begin{cases} \text{Unicast} \\ \text{Multicast} \\ \text{Broadcast} \end{cases}$

Functions :- Framing, Physical Addressing, Error Control, Flow control, Access control.

## 3) Network layer :-

- decides where the data should go using IP addresses.
- transmission of data from one host to the other located in diff. location.

Functions :- Routing, logical Addressing (IPaddr.)

## 4) Network layer :-

- Ensures reliable delivery of data.



- provides services to application layer and takes services from network layer.
- data in transport layer is referred as segments.
- provides acknowledgement of successful data transmission and re-transmits data if an error is found.

uses TCP/UDP Protocol.

↓      ↓  
reliable    fast

Function:- Segmentation & Reassembly, Service Point Addressing.

### 5) Session layer:-

- responsible for establishment of connections, management & termination of sessions b/w two devices.
- it also provides authentication & security.

Function :- Session Establishment, maintenance & termination, synchronization.

Synchronization → allows a process to add checkpoints that are considered as synchronization points in data. These points help to identify error so data is re-synchronized properly and data loss is avoided.

Dialog Controller:- allows two system to start communication with each other in half duplex or full duplex.

eg:- Staying logged into a website until you log out.

6) Presentation layer:-

→ also called translation layer.

- uses TLS / SSL protocol.

Function :- Translation, Encryption / Decryption

(encrypted data known as cipher text)

↓  
C (decrypted data known as plain text)

compression  $\rightarrow$  reduces no. of bits that need to be transmitted on n/w.

### 7) Application layer:-

→ closest to user → where apps communicate with network.

function: File Transfer, Mail services.

### TCP/IP Model:-

→ a framework for how data moves across the internet.

→ It has 4 layers.

- It has 4 layers.
- TCP/IP model is simpler and commonly used in today's Internet and networking systems.

OSI

Application layer  
Presentation layer

session layer

Transport layer

Network layer

Data Link Layer

physical layer

## TCP/IP

Application layer

Transport layer

Internet hayer

network Access Layer.

- 1) Physical layer / Network Access layer / Link layer:-
- deals with actual physical connection b/w devices on some local n/w like computer connected by cables or communicating through wifi.
  - this layer makes sure that data can travel over the hardware such as wires, switches or wireless signals.
  - uses MAC addr. to identify devices.

- 2) Internet layer:-
- decides where the data goes (routing).
  - finds best path for data to travel across ~~XXXXXX~~ diff. networks so it can reach the right destination.
  - This layer uses Internet Protocol (IP) to give every device a unique IP addr., which helps identify where data should go.

- 3) Transport layer:-
- responsible for making sure that data is sent reliably and in correct order b/w devices.
  - uses TCP / UDP protocols.

- 4) Application layer:-
- where apps interact with n/w.
  - supports diff. protocols like HTTP / HTTPS, SMTP, FTP & DNS.
  - it also manages encryption / decryption to keep data safe and session management to keep track of ongoing connections.



When you send data:

- Your device uses IP addr. to figure out where to send packet.
- on the local n/w, the router uses the MAC addr. to deliver it to right device.

MAC addr. → \* uniquely identify a device on a local network.  
(48 bit)

- \* assigned by manufacturer.
- \* permanent.

IP addr. → \* addr. assigned to a device by a network or ISP.

IPv4 (32 bit) or IPv6 (128 bit)

- \* can be changed, when device connects to diff. network.

IP addr. serves as an identifier that allows device to send & receive data over the n/w, ensuring data reaches the correct destination.

### Subnet:-

→ Subnet (sub-network) is a slice of bigger IP addr.

For e.g. In a company, diff. departments can each have their own subnet, keeping their data traffic separate from others.

→ Subnets makes the network faster and easier to manage and also improves.

- \* subnet creates smaller, isolated groups inside a big n/w.

### Why do we use subnets:-

- Organize the network → easier to manage.
- Improve performance → reduce unnecessary traffic
- Increase security → you can isolate sensitive systems.

→ Efficient use of IP addresses → avoid

Every device in a n/w has an IP addr.

It has two parts:

- 1) Network Part: tells which n/w the device belongs to.
- 2) Host Part: tells the specific device.

→ A subnet mask is used to separate the n/w part from the host part.

### Data routing:

When a device sends info. to another device over the internet -

→ The data is divided into packets.

→ Each packet contains the IP addr. of the device it is destined for.

→ Routers within the network read the destination IP addr. on each packet and determine the best path for the packet to travel.

Routers communicate with each other to update & maintain records of the fastest and most efficient routes for data.

For devices on diff. n/w data must travel through multiple routers across the internet.

Each router makes independent decisions about the best route for the packets based on destination IP addr.

### Need for Classful Addressing:-

→ Simplified IP Allocation.

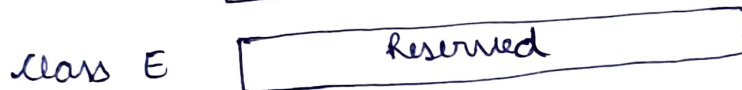
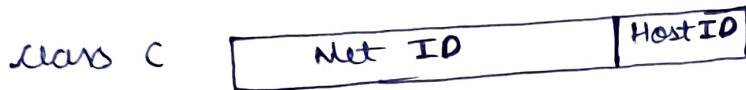
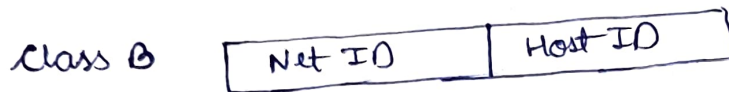
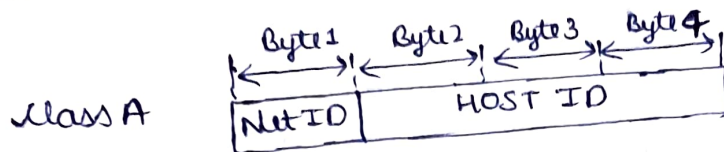
→ Faster Routing

→ Scalability

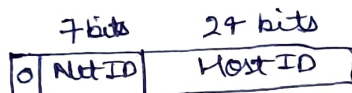
→ Interoperability

## Classes of IP Addressing:-

32 bit IP addr. is divided into 5 classes



### 1) Class A:-



In class A      Net ID  $\rightarrow$  8 bits  
                         Host ID  $\rightarrow$  24 bits

IP range

0.0.0.0 -  
127.255.255.255

So to calculate no. of hosts

$$2^{24} - 2 = 16,777,214 \text{ addr.}$$

Default Subnet mask = 255.0.0.0

First bit of class A is set to 0

Total no. of net. addr.  $2^7 = 128$

### 2) Class B:-

In class B

Net ID  $\rightarrow$  16 bits  
Host ID  $\rightarrow$  16 bits

$\rightarrow$  IP addr range

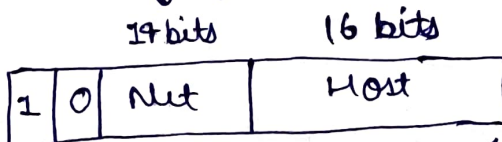
128.0.0.0 -  
191.255.255.255

$$\text{No. of Host addr} = 2^{16} - 2 = 65534$$

First two bits of first octet is 10.

$\rightarrow$  Default Subnet mask

255.255.0.0



$$\text{Total no. of network addr.} = 2^{14} = 16384$$



### b) Class C:-

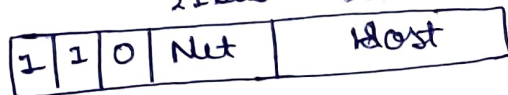
IP addr. belonging to class C are assigned to small sized networks.

Net ID  $\rightarrow$  24 bits

Host ID  $\rightarrow$  8 bits

$$\text{Total no. of Host addr.} = 2^8 - 2 = 256 - 2 = 254$$

First 3 bits of octet 1 in class C are 110



$$\text{Total no. of net. addr.} = 2^{21} = 2097152$$

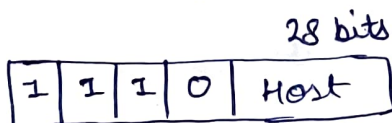
$\rightarrow$  IP range 192.0.0.0 — 223.255.255.255

$\rightarrow$  Default subnet mask  $\rightarrow$  255.255.255.0

### c) Class D:-

IP addr. of class D are reserved for multicasting.

First 4 bits of octet 1 is 1110.



$\rightarrow$  remaining bits identifies diff. multicast groups.

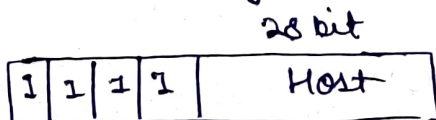
$\rightarrow$  multicast group is not tied to a particular network or set of hosts instead, hosts can join or leave group dynamically.

$\rightarrow$  class D does not possess any subnet mask.

$\rightarrow$  IP addr range — 224.0.0.0 — 239.255.255.255

### e) Class E:- IP addr. are reserved for experimental & research purposes,

$\rightarrow$  IP addr. range  $\rightarrow$  240.0.0.0 — 255.255.255.255



$\rightarrow$  Do not have any subnet mask.

## CIDR (Classless Inter Domain Routing):-

CIDR is a notation (IP/prefix) and a system that replaced old class-based addressing, making IP addr. allocation and routing more flexible & efficient.

e.g. 192.168.1.0/24

convert or find subnet mask of given cidr range:-

/8  $\rightarrow$  255.0.0.0

/16  $\rightarrow$  255.255.0.0

/24  $\rightarrow$  255.255.255.0

/32  $\rightarrow$  255.255.255.255

Subnet mask for 172.16.130.45/15

11111111.11111110.00000000.00000000

/15 means 15 n/w bits

so host bits =  $32 - 15 = 17$  bits.

Subnet mask will be - 255.254.0.0  
11111111.11111110.00000000.00000000

so 255.254.0.0 is subnet mask.

No. of usable host addr. =  $2^{17} - 2 = 131072 - 2 = 131070$

Every IP addr. is 32 bit long, grouped into 4 octet each 8 bits.

For example:

IP = 172.16.130.45

In Binary

172  $\rightarrow$  10101100

16  $\rightarrow$  00010000

130  $\rightarrow$  10000010

45  $\rightarrow$  00101101

Full IP  $\rightarrow$  10101100.00010000.10000010.00101101

$\rightarrow$  Convert CIDR to subnet :-

IP/n = first n bits are 1 (n/w), rest are 0 (host)

Suppose with CIDR 172.16.130.45/20

Subnet mask 11111111.11111111.11110000.00000000

$\rightarrow$  255.255.240.0

$\rightarrow$  Block size trick:

Find the first octet that is not 255 in subnet mask.

Block size = 256 - that octet  
 $= 256 - 240 = 16$

$\rightarrow$  Find n/w address :-

IP = 172.16.130.45/20

Block size = 16

Interesting octet = 130

$130 \div 16 = 8$  round down,  $8 \times 16 = 128$

So, net ~~20~~ = 172.16.128.0



→ Find Broadcast addr.

Start with network address, add block size - 1 in the interesting octet and set all later octets to 255.

$$128 + 16 - 1 = 143$$

$$\text{Broadcast addr.} = 172.16.143.255$$

usable range:-

$$\text{First usable} = \text{n/w} + 1 = 172.16.128.1$$

$$\text{last usable} = \text{broadcast} - 1 = 172.16.143.254.$$

$$\text{Host bits} = 32 - 20 = 12 \text{ bits}$$

$$\begin{aligned} \text{Total addresses} &= 2^{12} - 2 = 4096 - 2 \\ \text{which are usable} &= 4094 \end{aligned}$$

DNS (Domain Name System):-

- DNS is the internet phonebook.
- It translates human names like `www.example.com` into IP addr. that computer uses to find each other.

translates domain names into IP addresses.

DNS ensures that the request reaches the correct server by resolving the domain to its corresponding IP addr.

## How DNS works:-

### 1) User Input:-

- you type `www.gdg.org` into your browser.
- Your computer needs IP addr. to actually connect, for this DNS helps.

### 2) Local Cache Check:-

- First your browser checks its own memory (cache) "Have I recently visited this domain or not?"
  - if found → it uses that IP directly (faster)
  - if not found → it asks the OS DNS cache
  - if still not found - then next step is -

### 3) DNS resolver query:-

- Now your computer asks a DNS resolver (like a helper)
- This resolver is usually run by your ISP. (Airtel, Jio, BSNL) or a public resolver like Google DNS (8.8.8.8) or Cloudflare (1.1.1.1)
- The resolvers job is to go out and find the correct IP for you.

### 4) Root DNS Server:-

- If resolvers doesn't already have the answer in its cache, it first asks a Root DNS server.
- There are only 13 main root server clusters worldwide.
- Root servers don't know the final IP of your site but they know where to look next.

eg. for `www.gdg.org`, the root server says this is a .org domain, so ask .org TLD servers.

### 5) TLD (Top Level Domain):-

→ TLD servers handle extensions like .com, .org, .net, .in.

→ the .org server is asked "where can I find gfg.org?"

→ the .org TLD replies: "go to this authoritative DNS server for gfg.org."

DNS

### 6) Authoritative Server:-

→ this is the final source.

→ the authoritative server for gfg.org has the real DNS records (A, AAAA, CNAME, MX etc).

→ it finds the A record → which maps

www.gfg.org → 1.2.3.4

It replies the resolver with the IP.

### 7) Final response to your computer:-

→ the resolver now gives the IP addr. back to your computer.

→ your browser uses this IP to connect directly to website server.

The page then loads.

### Note

→ Each DNS answer comes with a TTL (Time to Live)

→ This tells how long the answer can be kept in cache.

→ Next time you visit the site, the resolver can reply instantly without asking root/TLD/authoritative servers again.



## NAT (N/w Address Translation):-

Every device (like your laptop, phone or PC) needs an IP addr. to communicate on the internet.

- Public IP addresses (unique worldwide) are limited in number.
- Private IP addresses are not directly accessible on internet.

So if your home has 5 devices connected to wifi, you don't get 5 public IPs from your ISP. You usually get 1 public IP.

But then how do all devices in your home access the internet using that 1 public IP?

This is where NAT comes

NAT is a technique where the router translates private IP addresses inside your local n/w into a single public IP address.

NAT acts as middleman b/w your devices and the outside internet.

### How NAT works:-

- 1) Device sends request → your laptop (192.168.1.0) wants to open google.com.
- 2) Router with NAT receives it → router sees the request notes down:-  
Source Private IP addr. → 192.168.1.0  
Source Port: 1234

- 3) Router Translates  $\rightarrow$  Router replaces the source private IP with its public IP (49.37.55.20) & assigns a unique port number.

Source public IP: 49.37.55.20

Port: 5678.

- 4) Server Responds  $\rightarrow$  Google's server replies to 49.37.55.20:5678

- 5) Router Translates back  $\rightarrow$  NAT look at its table, finds who requested using port 5678, & sends the reply back to 192.168.1.100

Why NAT is imp:-

- $\rightarrow$  Saves public IP addresses (since they are limited)
- $\rightarrow$  Provides security  $\rightarrow$  internal devices are hidden from outside world.  
(hides private IP)
- $\rightarrow$  Allows multiple devices to share one public IP.

Port Number Masking:-

Port masking (or port translation) means NAT device modifies the source port number in addition to IP addr. when forwarding a packet to the internet.

This ensures even if multiple devices use the same port, NAT can still tell them by assigning unique masked ports.

IP:- which device

Port:- which application.

lets say 2 devices want to visit google

laptop: 192.168.1.10 : 1234

phone: 192.168.1.20 : 1234

both device uses same port number inside the n/w.

If NAT just replace IP and kept port same, the internet would get confused.

So NAT does port masking :-

laptop: 49.37.55.20 : 5001

phone: 49.37.55.20 : 5002

Now on Internet side, both requests looks unique

49.37.55.20 : 5001 → NAT maps it to 192.168.1.10 : 1234

49.37.55.20 : 5002 → NAT maps it to 192.168.1.20 : 1234  
using its translation table.

Basically port no. are 16 bit values → range 0 to 65535

1) Well known ports (0-1023) → fixed for common services.

HTTP → 80

HTTPS → 443

SSH → 22

FTP → 21

DNS → 53

2) Ephemeral (Dynamic Ports) → (1024-65535)  
temporary ports chosen by your device when making connections.

∴ Private IPs are only unique within one local n/w.  
Within the whole internet private IPs can repeat  
in diff. local n/w.



### Case 1

If there is no NAT and devices have only private IP → no public IP is assigned and they can not access the internet because private IPs are not valid ~~outside~~ on internet.

### Case 2

If there is no NAT in router but ISP gives public IPs to every device → they do ~~go to~~ get ~~state~~ public IPs, they can reach to internet but

- wastes many public IP.
- devices are exposed directly to internet.
- requires firewall rules for protection.

When your laptop, phone or TV connects to your wifi/router, they need IP addr. to communicate in the local n/w.

This IP is usually a private IP.

These private IPs are given by a system called DHCP (Dynamic Host Configuration Protocol), which run on your router and are assigned automatically.

If DHCP isn't available, you can set IP manually or OS gives itself a temporary one.

ISP → gives public IP to your router.

DHCP → gives private IP to devices.

NAT → uses public IP from ISP to let multiple private IP devices talk to the internet at the same time.

## TCP (Transmission Control Protocol):-

→ a connection oriented transport layer protocol, that ensures reliable, ordered and error-free delivery of data b/w application over a n/w.

Before sending data TCP creates a connection using 3-way handshake.

SYN → SYN-ACK → ACK

### 3 way handshake:-

Imagine 3 way handshake as making a phone call.

Step 1: SYN (Synchronize): You dial someone  
→ Hello, can you hear me? I'd like to talk.

Step 2: SYN-ACK: They respond  
→ Yes I can hear you clearly, can you hear me too?

Step 3: ACK: You confirm  
→ Perfect I can hear you too. Let's start our conversation.

### Actual

Step 1: SYN (Client to Server).  
Client says: I want to connect.

Sends SYN flag = 1

Includes initial sequence no. 100

→ I'll start numbering my data from 100.

Step 2: SYN-ACK (Server to Client)

Server says: I am ready too.

Sends SYN flag = 1 AND ACK flag = 1

Acknowledges client's sequence no  $(100 + 1) = 101$

sends its own sequence no 300

I got your 100, expecting 101 next. I'll start from 300.

Step 3: ACK (client to server):

Client says: great let's start

Sends ACK flag = 1

Acknowledges server's sequence number ( $300 + 1 = 301$ )

I got your 300, expecting 301 next.

Connection established.

→ TCP protocol ensures all data reaches destination  
If packets are lost, TCP resends them.  
If packets arrive out of order, TCP rearranges them.  
Every packet has checksum to check errors.

→ TCP is slower as compared to UDP because it  
spends time checking and ensuring reliability.

Used in critical ~~applications~~ applications like

→ web browsing → loading a web page must be  
100% correct.

→ Emails (SMTP, IMAP, POP3) → you can't afford  
missing parts of an  
email.

→ File transfer (FTP, SFTP → even 1 missing byte  
can corrupt the file)



## UDP (User Datagram Protocol) :-

→ a simple, connectionless and lightweight transport layer protocol that enables fast, low-latency data exchange over Internet Protocol suite.

→ UDP prioritizes speed over reliability.

→ Data is sent in small, independent packets called user datagram.

each datagram contains source and destination port number in its 8 byte header to identify the sending and receiving application.

## common application:-

→ video & audio streaming.

→ VoIP (Voice over IP): enables realtime voice communication.

→ Online gaming

→ (Zoom, Skype)

→ DNS lookups.

## DHCP (Dynamic Host Configuration Protocol) :-

→ a network service that automatically assigns IP addresses and other n/w settings (default gateway, DNS) to devices when they connect to a n/w.

without DHCP, you have to manually enter an IP addr. on every phone, laptop etc.

## What DHCP give:

Subnet mask → to understand your local n/w range.

Default gateway → routers IP to reach the internet.

DNS server → to translate domain names into IPs.

Lease time → how long you can keep this IP.

When DHCP gives you an IP addr., it's not permanent. It's like renting a hotel - you don't own it forever.

This temporary ownership is called a lease.

The IP addr. assigned by DHCP can be reused by another device once:

→ You disconnect or

→ Your lease time expires

How DHCP works:- (DORA process)

1) D - Discover:-

Device says "Hey is there any ~~DHCP~~ DHCP server?  
I need an IP addr."

2) O - Offer:-

DHCP server replies: "Yes I can offer you IP x.x.x.x,  
subnet mask 255.x.x.x, gateway,  
DNS."

3) R - Request:-

Device ~~DHCP~~ responds: "Okay, I'd like to take that IP.  
Please reserve it for me".

4) A - Acknowledge:-

DHCP server confirms: "Great x.x.x.x is now  
officially yours".

Now your device can communicate on the n/w &  
internet.

## Internet Protocol (IP):-

- a rulebook that make sure every device on the internet can be uniquely identified and can send/receive data packets properly.
- IP works at the Network layer of OSI model.
- without IP your data wouldn't know where to go.
- IP decides the path your data takes across routers to reach destination.

## Monitoring & Troubleshooting Tools:-

- ping, traceroute → paths of packets.
- nslookup, dig → DNS troubleshooting.
- netstat, ss → open ports, connection.
- curl, telnet → testing connectivity.

Load Balancing → L4 → TCP/UDP  
L7 → HTTP/HTTPS

## Proxy:-

In CN, Proxy is simply a middleman b/w a client and the server.

- Instead of your device directly talking to a server, the proxy sits in b/w and forwards requests/response.

### this proxy can:-

- Hide your real ip addr.
- Filter or control access.
- Load balance or provide security.



In networking terms:-

Client sends a request → Proxy server forwards it  
→ to destination server.

Destination server replies to → proxy server → client.

### Types of Proxy:-

#### 1) Forward Proxy:- (Client side proxy)

→ Forward proxy sits b/w client and the internet.  
Clients configure the proxy and ask it to fetch  
resources on their behalf.

→ Forward proxy is client side helper. (caching, content  
filtering)

#### Use cases:-

- corporate internet filtering (block/allow sites)
- caching web content to save bandwidth (Squid).
- anonymity for clients.
- Bypassing geo-restriction.

Eg:- Squid, a configured HTTP proxy in browsers.

#### 2) Reverse Proxy (server side proxy):-

→ reverse proxy sits in front of one or more servers  
and accepts requests from the internet,  
then forwards them to appropriate backend server.

→ server side front door (load balancing, SSL  
termination, WAF).

### use cases :-

- distribute incoming requests to multiple backend servers.
- SSL/TLS termination.
- caching static content.
- web application firewall (WAF) + request filtering.
- Hiding real backend IPs.

example - Nginx, Apache ~~httpd~~ acting as reverse proxy, Cloudflare

### SSL/TLS Termination (Secure Socket Layer / Transport Layer Security)

SSL/TLS encrypt data b/w client and a server.

e.g. - when you see https:// instead of http://, TLS is protecting the connection.

- SSL/TLS termination means the point where encrypted ~~data~~ HTTPS traffic is decrypted into plain ~~at~~ HTTP.
- It happens at special device or software (like load balancer, reverse proxy or firewall) before the traffic reaches your actual web server.

\* The load balancer does the heavy work of encryption / decryption.

The backend server just handle plain traffic (HTTP), which is lighter and faster for them.

## Why do we use SSL/TLS termination :-

### 1) Performance (offloading) :-

Encryption / Decryption is CPU heavy.  
Instead of every backend server doing it, a dedicated device (load balancer) does it once.

### 2) Centralized Certificate Management

### 3) Scalability :-

easier to scale backend server because they don't need SSL config.

### 4) Inspection / Security Tools :-

Since traffic is decrypted at termination point, security tools can inspect it more easily.

## variations :-

### 1) SSL/TLS termination :-

decrypts at load balancer → forwards plain HTTP to backend.

### 2) SSL/TLS Passthrough :-

load balancer does not decrypt. Encrypted traffic goes straight to backend, which decrypts.  
(more secure end-to-end, but heavier on server)

### 3) SSL/TLS bridging (re-encryption) :-

load balancer decrypts, inspects, then re-encrypts before sending to backend.



## SSL/TLS Certificate:-

a digital document that binds a website's identity (domain, org) to a public key and is signed by a trusted certificate authority (CA).

Browsers use it to verify, you're really talking to the right server and to build an encrypted connection.

## How it is issued:-

- You generate a private key and a CSR (Certificate Signing Request) that contains the public key + identity info.
- You send CSR to CA (Certificate Authority).
- CA validates the request.
- CA issues a signed certificate you install on your server.

## Load Balancing:-

→ distributing incoming client requests across multiple backend servers to improve capacity & availability.

### Basic algorithms:-

- 1) Round robin → sends request sequentially across servers.
- 2) Least connection → send to the server with fewest active connections.
- 3) Weighted → send more traffic to more powerful servers.
- 4) IP hash / sticky session → send same client IP to same backend.

## Firewalls :-

Firewall is like security guard at the entrance of your building (network).

- it checks who is coming in or going out.
- decides whether to allow or block the traffic based on rules.

Firewalls can be stateless or stateful.

### Types

#### 1) Stateless firewalls :-

- it checks each packet individually, without remembering anything about previous packets.
- it doesn't care if packets are part of an existing conversation.

You have to write explicit rules <sup>for</sup> both inbound & outbound ~~traffic~~ direction.

some as AWS NACLs

#### 2) Stateful firewalls :-

- it is smart, it keeps track of connections (state).
- it remembers / knows whether a packet belongs to an existing conversation.

If you allowed ~~outbound~~ <sup>inbound</sup> traffic, return ~~inbound~~ traffic is automatically allowed. no separate rules.

more secure but needs more memory & CPU.

some as AWS security groups.

## VPN (Virtual Private Network) :-

- a technology that creates a secure, encrypted connection b/w your device and other n/w over the internet.

It allows you to send and receive data securely, as if you were directly connected to that private network, even if you're actually using a public n/w.

### without VPN:-

- your data travels openly over the internet.
- Hackers, ISPs or even government can see what you're doing.
- your IP addr. is visible.

### with VPN:-

- your data is encrypted.
- your real IP is hidden
- you can bypass restrictions (like accessing content blocked in your country).

### How VPN works:-

- You connect to the internet normally. (through ISP)
- You turn on ~~to~~ the VPN app → it connects you to a VPN server.
- Encryption starts → all your internet traffic is scrambled using encryption protocols.
- VPN server forwards your request to the internet.  
\* the website sees VPN server's IP, not your real IP.
- Response comes back → website sends data to VPN server → VPN server encrypts it → sends it securely back to you.



## VPN Key features:-

- 1) Encryption. → Protects your data from hackers.
- 2) Anonymity → Hides your real IP addr.
- 3) Remote access → Lets employee securely access company resources from anywhere.
- 4) Bypass geo-blocks → watch content available in another country.
- 5) Public Wifi security → keeps you safe on insecure networks.

## VPN protocols:-

- PPTP (Point to Point Tunneling Protocol) → old, fast, less secure.
- L2TP/IPsec (Layer 2 Tunneling Protocol) → secure but slower.  
with IPsec
- OpenVPN → Very secure, widely used, open source
- WireGuard → New, faster, & more efficient protocol.